

Web Programming Lab

JavaScript Basics

Week 5



Hello, World!

```
<!-- index.html -->

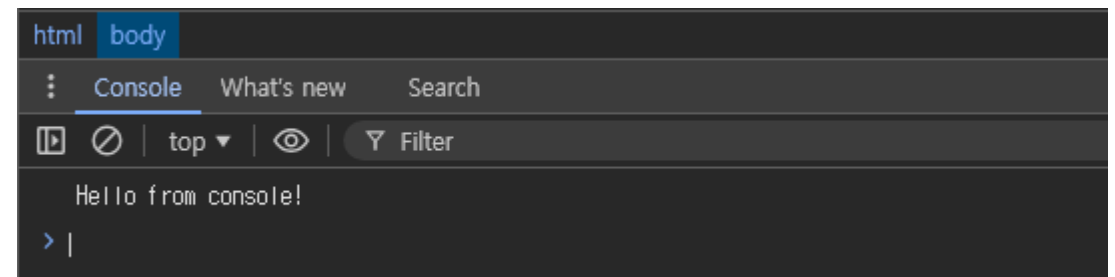
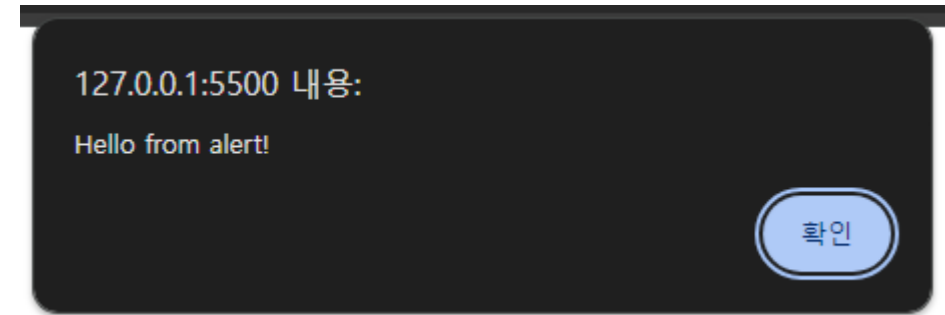
<!DOCTYPE html>

<body>
  <h1>Nothing here</h1>
</body>
<script src="./script.js"></script>
```

```
// script.js

"use strict";

alert("Hello from alert!");
console.log("Hello from console!");
```





Declaring Variables

- 3 (or 4) ways to declare variables
- Recommend to use let and const only

Declarator	Can reassign?	Can re-declare?	Scope	Hoisting *	Note
let	Yes	No	Enclosing block {...}	No	<i>Preferred</i>
const	No §	No	Enclosing block {...}	No	<i>Preferred</i>
var	Yes	Yes	Enclosing function, or global	Yes, to beginning of function or file	<i>Legacy, beware its quirks, try not to use</i>
None (implicit)	Yes	N/A	Global	Yes	<i>Forbidden in strict mode</i>



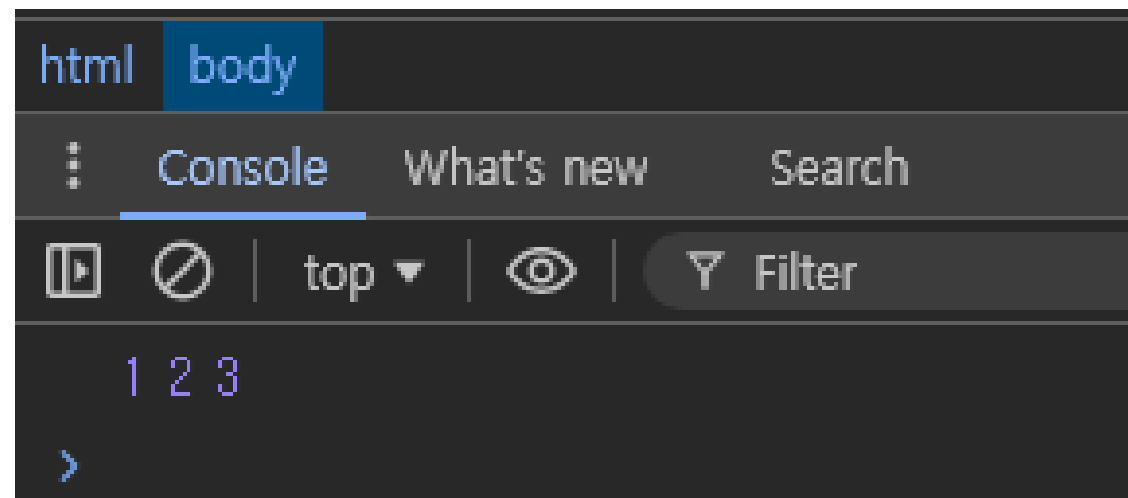
Declaring Variables

```
// script.js

"use strict";

// declaration
let a = 1;
const b = 2;
var c = 3;
// d = 4;

// alert(a + ", " + b + ", " + c);
console.log(a, b, c);
```





Declaring Variables

```
// script.js

"use strict";

// declaration
let a = 1;
const b = 2;
var c = 3;

// reassign
a = 11;
b = 22;           // invalid
c = 33;

// re-declare
let a = 111;      // invalid
const b = 222;    // invalid
var c = 333;
```

Declarator	Can reassign?	Can re-declare?
let	Yes	No
const	No [§]	No
var	Yes	Yes
None (implicit)	Yes	N/A

```
JS script.js > ...
1  "use
2
3  let
4
5  let a = 11;

Cannot redeclare block-scoped variable 'a'. ts(2451)
let a: number
View Problem (Alt+F8) No quick fixes available
```



Scope

```
// script.js

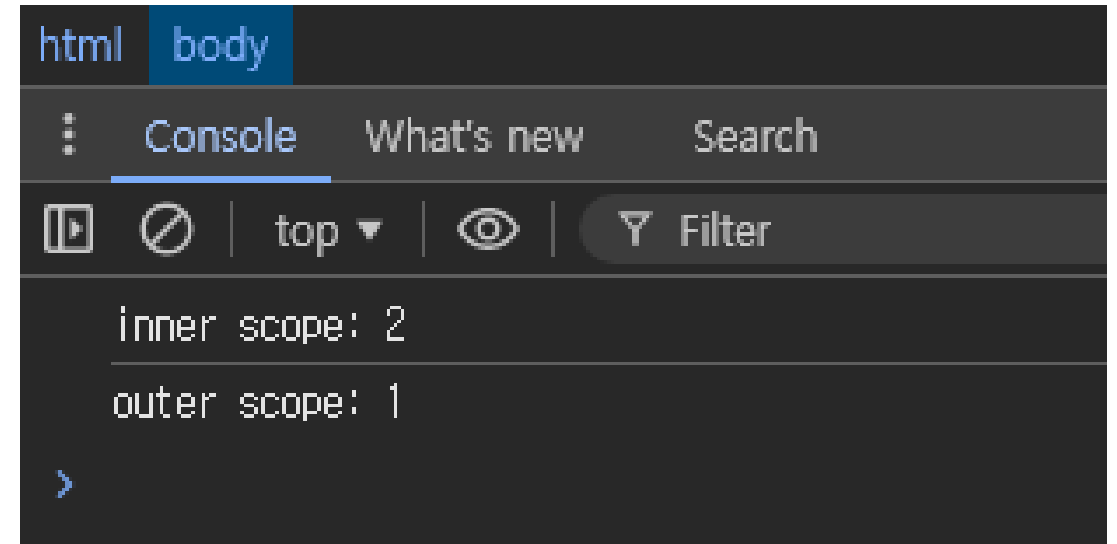
"use strict";

// declaration
let a = 1;

// re-declare
let a = 11;    // invalid

{ // new scope
  let a = 2; // valid!
  console.log("inner scope: " + a);
}

console.log("outer scope: " + a);
```





Scope

```
// script.js

"use strict";

// declaration
let a = 1;

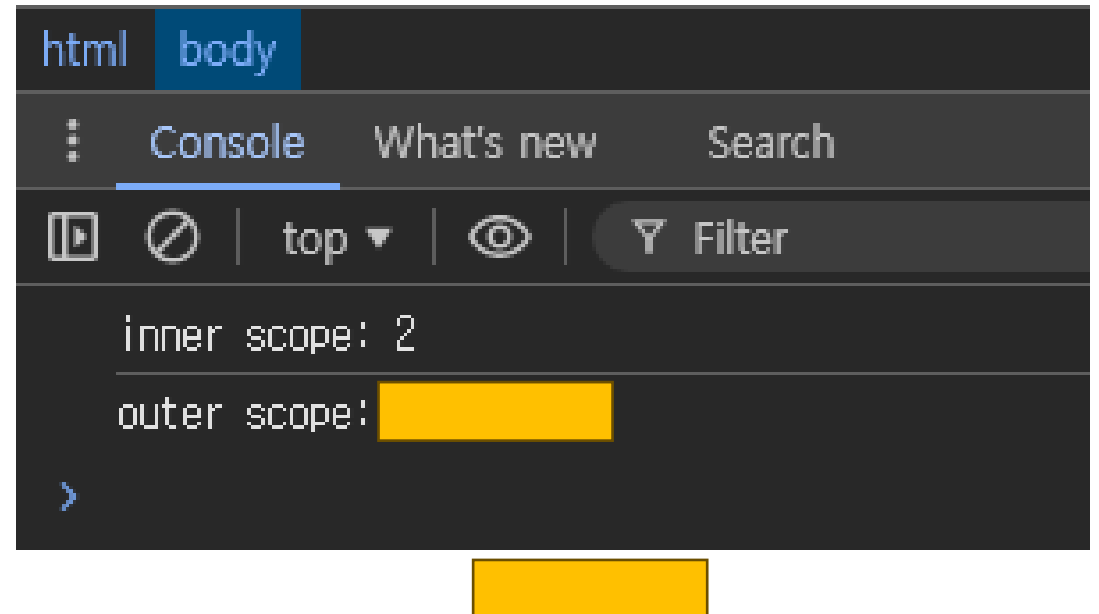
// re-declare
let a = 11;    // invalid

{ // new scope
  let a = 2; // valid!
  console.log("inner scope: " + a);

  a = 3;
}

console.log("outer scope: " + a);
```

- What will be printed?

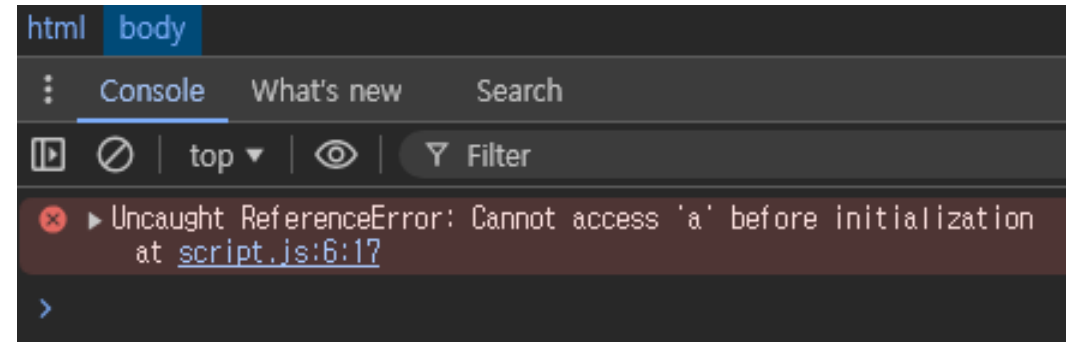




Hoisting

- Hoisted variable: considered as it is declared at the top-most of its scope
- Q. Variable with let and const keyword are not hoisted... right?
 - A. ...

```
"use strict";  
  
let a = 1;  
  
{  
    console.log(a);  
    let a = 2;  
}
```





Hoisting

```
// script.js

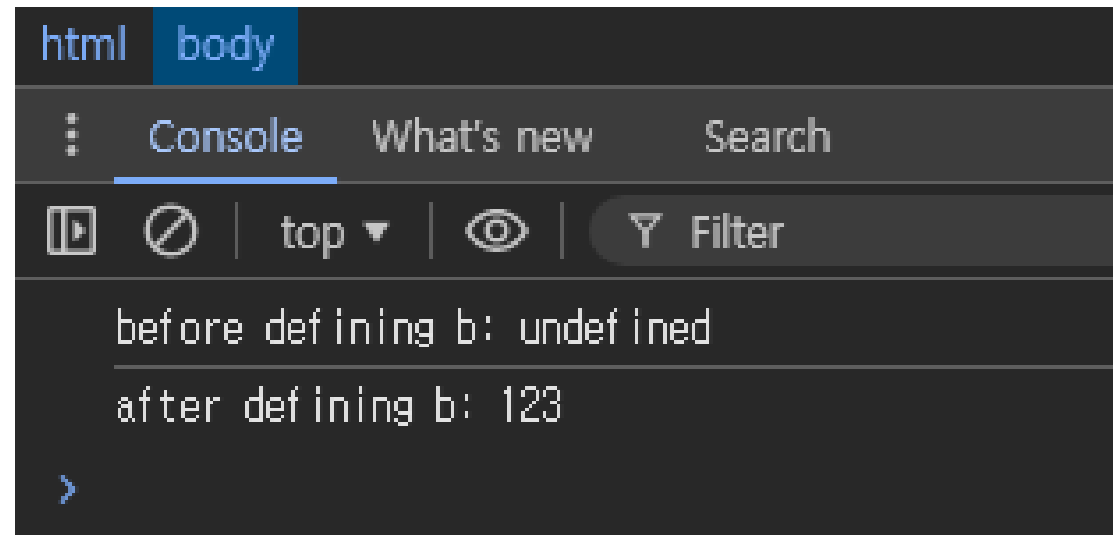
// CAUTION: NOT RECOMMENDED

"use strict";

console.log("before defining b: " + b);

// declare and define b
var b = 123;

console.log("after defining b: " + b);
```





Hoisting & Scope

```
"use strict";

console.log("1: " + a);
console.log("2: " + b);

let a = 11;
var b = 22;

{
  let b = 33;
  console.log("3: " + a);
  console.log("4: " + b);
  a = 44;
  b = 55;
}

console.log("5: " + a);
console.log("6: " + b);
```

- Which line(s) raise error?
- After removing error lines, what will be printed?



Conditional Statements

```
"use strict";

let n = 42;

if (n % 15 == 0) {
    console.log("fizzbuzz");
} else if (n % 3 == 0) {
    console.log("fizz");
} else if (n % 5 == 0) {
    console.log("buzz");
} else {
    console.log(n);
}
```

```
"use strict";

let x = 10;
let y = 7;
let op = "+";

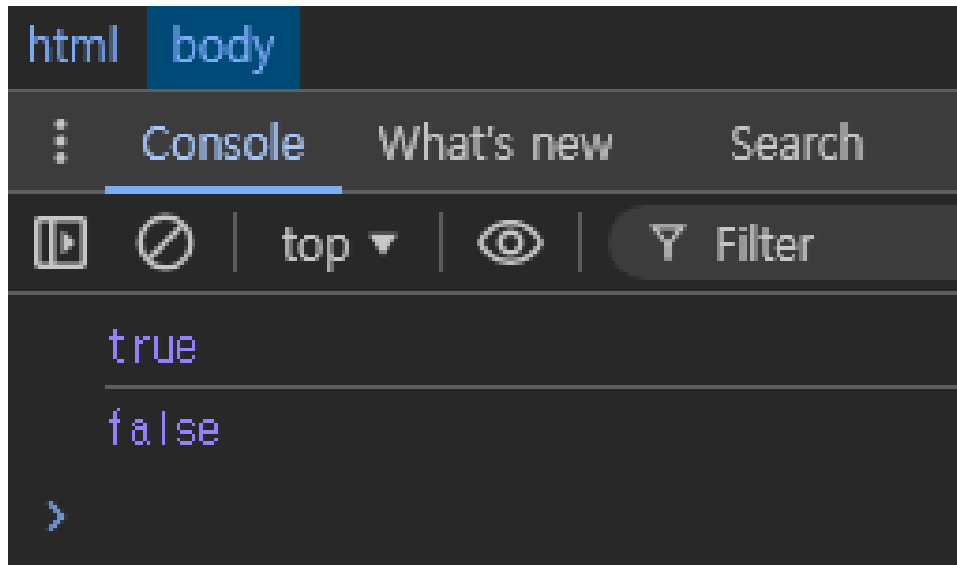
switch (op) {
    case "+":
        console.log(x + y);
        break;
    case "-":
        console.log(x - y);
        break;
    default:
        console.log("unknown op");
}
```



Equal vs Strict Equal

```
"use strict";
```

```
console.log(0 == "0");  
console.log(0 === "0");
```





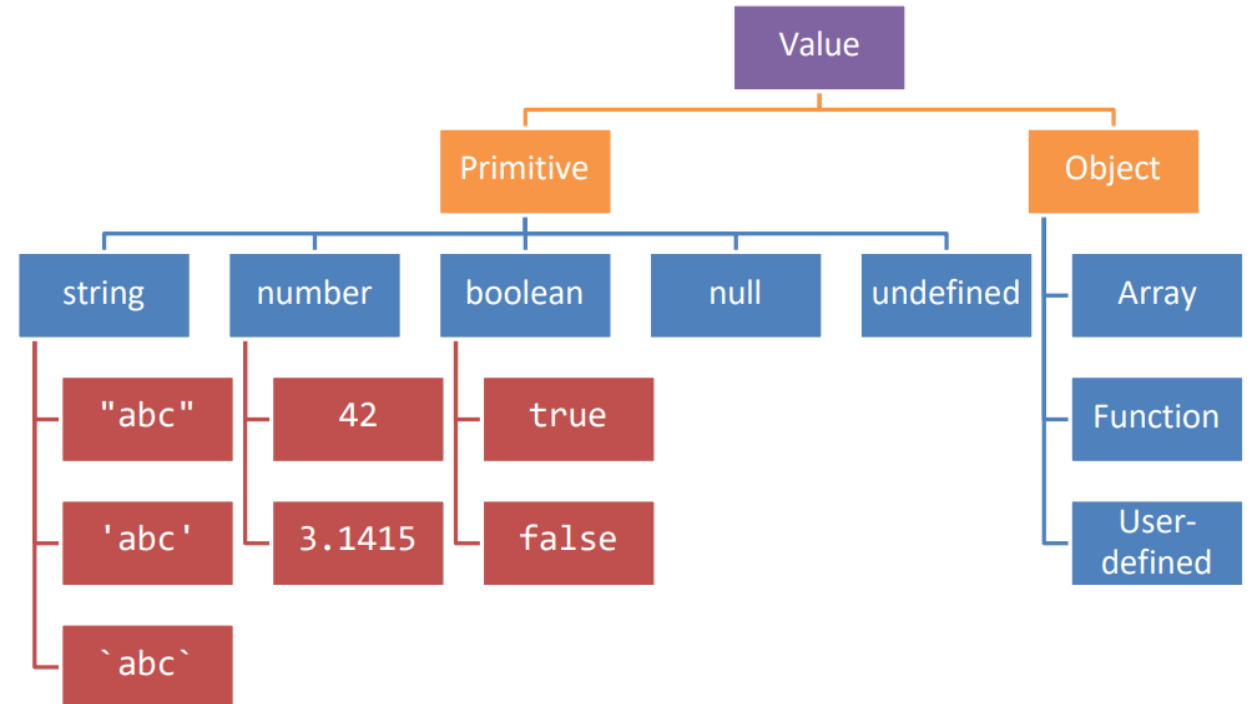
Primitive vs Object

```
"use strict";

let a = [1, 2, 3];
let b = [1, 2, 3];
let c = a;

console.log(a === b);
console.log(a === c);
console.log(b === c);
```

```
html body
⋮ Console What's new Search
⏮ ⏭ top 🔍 Filter
false
true
false
>
```





Loop Statements

```
"use strict";

for (let i = 1; i <= 9; i++) {
  for (let j = 1; j <= 9; j++) {
    console.log(i + "*" + j + "=" + i*j);
  }
}
```

8*9=72

9*1=9

9*2=18

9*3=27

9*4=36

9*5=45

9*6=54

9*7=63

9*8=72

9*9=81



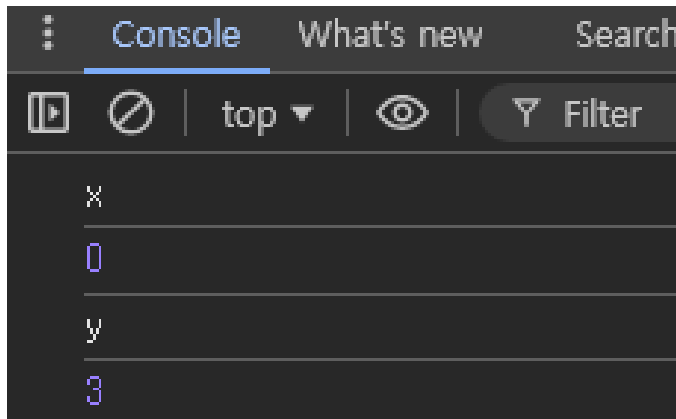


Loop Statements

```
"use strict";

let obj = {x: 1, y: 3};

for (let a in obj) {
  console.log(a);
  console.log(obj[a]);
}
```

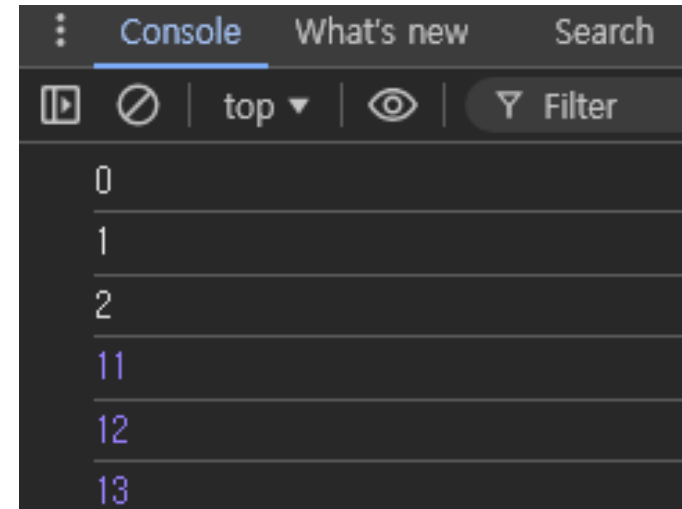


```
"use strict";

let arr = [11, 12, 13];

for (let a in arr) {
  console.log(a);
}

for (let a of arr) {
  console.log(a);
}
```



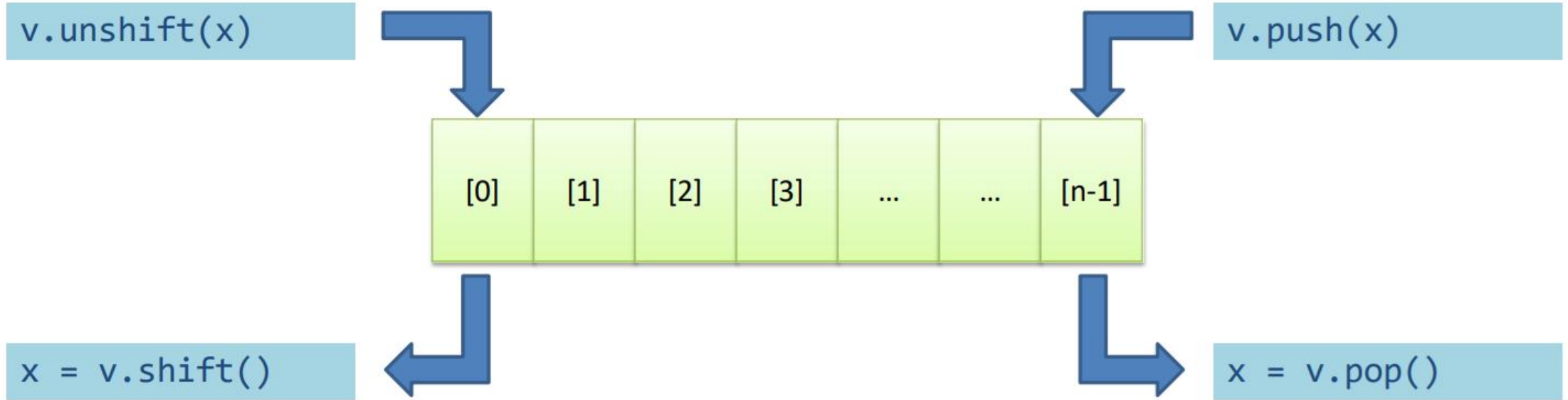


Array

- Container of many elements
- `let a = [ 1, 2, 3 ];`
- Supports lot of useful functions
- Array is object, not primitive
- Elements with different types are allowed



Array



Add/remove element at the beginning of the array

Add/remove element at the end of the array



Array

```
"use strict";

let arr = [1, 2, 3, 4, 5];
console.log("1: " + arr[2]);

let a = arr.shift();
console.log("2: " + a);

arr.push(6);
arr.push(7);
let b = arr.pop();
console.log("3: " + b);

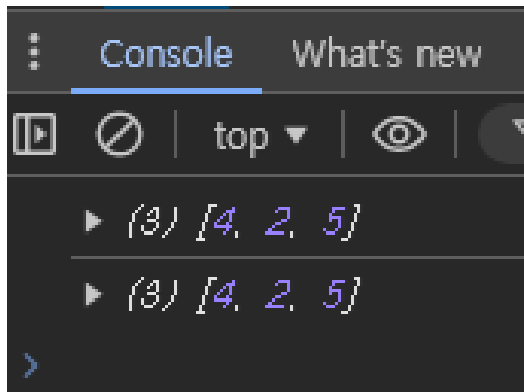
arr.unshift(8);
console.log("4: " + arr[1]);
```

- What will be printed?
- After each steps, what is the length of arr?
- After each steps, how arr looks like?



Array

```
"use strict";  
  
let arr1 = [1, 2, 3];  
let arr2 = arr1;  
  
arr1[0] = 4;  
arr2[2] = 5;  
  
console.log(arr1);  
console.log(arr2);
```



- Array is object, not primitive
- Which means, arr1 is passed as **reference** at line 4

